Software Development Management Lab · Lab 03

# Git & GitHub Workflow and Sprint Planning

## Working together on code, and planning your first sprint

**Ekramul Haque Amin**

Lecturer, AI and Data Science

Dhaka International University (DIU)

Course Title: Software Development Management | Course No: 0613-307

# Where We Are

LAB 1

**Project Charter**

What we build, and why

→

LAB 2

**Product Backlog**

User stories with MoSCoW priorities

→

LAB 3 · TODAY

**Code & Sprint Plan**

How the team works on code, and what goes into Sprint 1

**Today's question: how does a team of four build the same software without breaking each other's work, and decide what to build first?**

# Today’s Lab

PART 1

## Git & GitHub

- Why version control
- Git basics and key commands
- Branches and pull requests
- Merge conflicts

PART 2

## Sprint Planning

- Story points
- Planning poker
- Velocity and capacity
- A worked Sprint 1 plan

# Learning Objectives

**By the end of this lab, you will be able to:**

1. **Explain why teams use version control**
2. **Describe the basic Git workflow and commands**
3. **Explain branches, pull requests and merge conflicts**
4. **Estimate user stories with story points and planning poker**
5. **Build a sprint plan using velocity and capacity**

# Why Version Control? Sound Familiar?

```
project_final.zip
```

```
project_final_v2.zip
```

```
project_final_REAL.zip
```

```
project_final_REAL_fixed.zip
```

**Overwritten work: a teammate's copy replaces yours**

**No history: you can't go back to yesterday's version**

**No trace: who changed what, and why?**

> **Version control solves all three.**

# What Is Version Control?

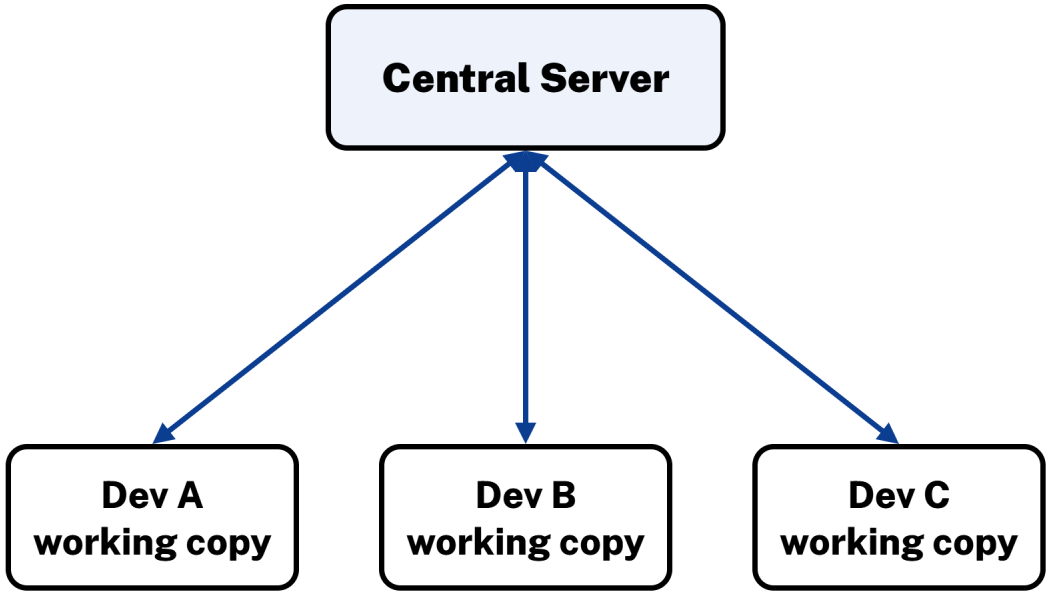
**A system that records every change to your files, so a team can work together and go back to any earlier version.**

- **Full history**
- **Teamwork**
- **Branching**
- **Undo mistakes**
- **Safe backup**
- **Who changed what**

**Git is the tool on your PC. GitHub is the website that hosts the shared copy.**

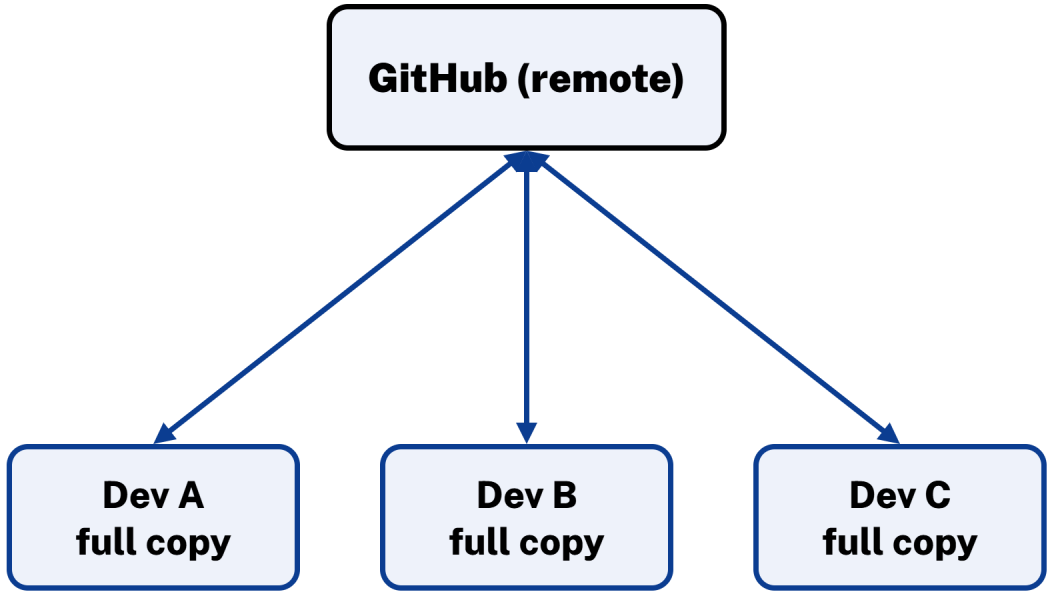
# Centralized vs. Distributed

## Centralized (e.g. SVN)




One server holds the history. Server down = no one can save.

## Distributed (Git)




Everyone has the full history. Work offline; sync later.

# Key Terms

**Repository**
The project folder plus its full history

**Commit**
A saved snapshot with a message

**Branch**
A separate line of work

**Merge**
Join one branch into another

**Clone**
Copy a remote repo to your PC

**Push / Pull**
Send changes up / bring changes down

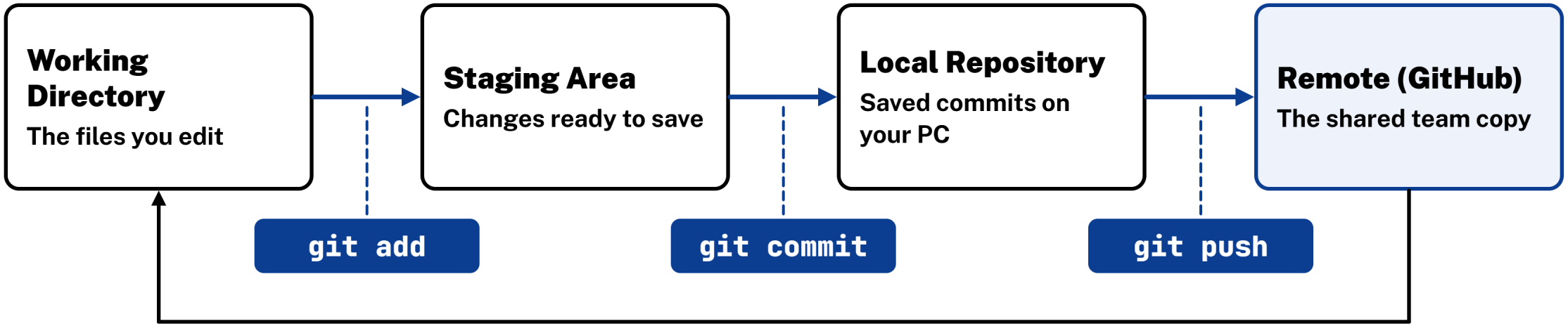
**Remote**
The shared copy on GitHub

**Pull Request**
A request to review and merge a branch

**Conflict**
Two changes to the same lines

# How Git Moves Your Work

**Every change travels through four places, one command at a time.**

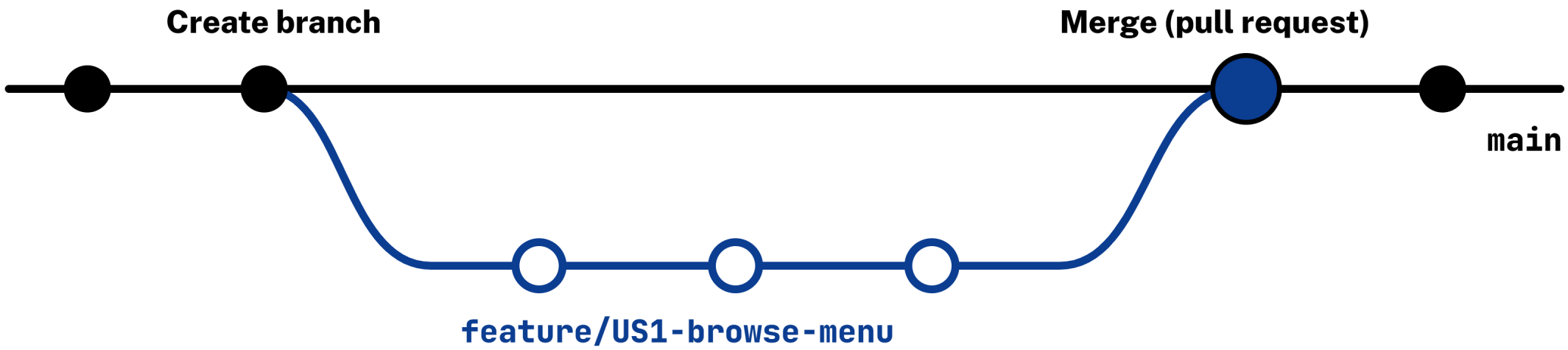



**git pull brings the team's latest changes back to you**

# Eight Commands You Will Use Every Day

| | |
|---|---|
| `git clone <url>` | Copy the team repo to your PC |
| `git status` | See what has changed |
| `git add <file>` | Stage changes for the next commit |
| `git commit -m "msg"` | Save a snapshot with a message |
| `git push` | Send your commits to GitHub |
| `git pull` | Get the team’s latest changes |
| `git switch -c <branch>` | Create a new branch and move to it |
| `git merge <branch>` | Join a branch into the current one |

# Branching: Work Without Breaking Main




> **main always holds working code. Each user story gets its own branch, then merges back after review.**

# The Feature-Branch Workflow

| | | |
|---|---|---|
| 1 | **Get the latest main** | `git pull` |
| 2 | **Create a branch** | `git switch -c feature/US1-browse-menu` |
| 3 | **Commit small steps** | `git add .` → `git commit -m "Add menu page"` |
| 4 | **Push the branch** | `git push -u origin feature/US1-browse-menu` |
| 5 | **Open a pull request** | On GitHub: compare your branch with main |
| 6 | **Review and merge** | A teammate reviews, then merges into main |
| 7 | **Delete the branch** | The work now lives in main |

# Pull Requests and Commit Messages

## Pull Request (PR)

A request to merge your branch into main, so a teammate can review it first.

**The reviewer checks:**

- It works and passes the tests
- It matches the user story
- The code is clear and readable
- No passwords or secret keys

## Commit Messages

**Weak**

```
update
```

```
fixed stuff
```

**Better**

```
Add menu search by category
```

```
Fix cart total after removing item
```

# Merge Conflicts: What They Look Like

**Two teammates changed the same line. Git cannot choose, so it asks you.**

```
<<<<<<< HEAD
delivery_fee = 50
=======
delivery_fee = 60
>>>>>>> feature/new-fee
```

Top part: your version (HEAD)

Bottom part: the incoming branch

Markers: <<<, === and >>> must be deleted

1. Find the markers → 2. Agree with your teammate → 3. Keep the right code → 4. Commit and test

# Git Golden Rules for Your Team

- Pull before you start work
- Commit small and often
- Write clear commit messages
- One branch per user story
- Never commit passwords or keys
- Never push broken code to main

# Part 2: From Backlog to Sprint

**Product Backlog**
All stories, ordered by priority (Lab 2)

→

**Sprint Planning**
Estimate, choose, split into tasks

→

**Sprint Backlog**
The stories and tasks for this sprint

**Sprint goal: one sentence saying what this sprint delivers**

**Key question: how much work fits in one sprint?**

# Story Points: Size, Not Hours

**A story point is a relative measure of how big a story is, compared with other stories.**

### Effort

How much work is there?

### Complexity

How hard is it to build?

### Uncertainty

How much is still unknown?

> **Like fruit: you know a watermelon is bigger than a mango without weighing either.**

# The Estimation Scale (Fibonacci)




**1–3: small and clear**

**5–8: medium to large**

**13: too big, split the story**

**?: need more information**

> **The gaps grow because big stories are harder to estimate precisely.**

# Planning Poker: How It Works

1. The Product Owner reads one user story aloud
2. The team asks questions about it
3. Everyone picks a card in secret
4. All cards are shown at the same time
5. Highest and lowest voters explain their reasons
6. Vote again until the team agrees

> **Secret votes stop people from simply copying the senior member.**

# Planning Poker: An Example

**US3: As a customer, I want to pay online so that I don't need cash.**

| Rahim | Sadia | Tanvir | Nusrat |
|---|---|---|---|
| 3 | 8 | 5 | 5 |
| LOWEST | HIGHEST | | |

**Sadia (8): "Two payment methods, and we must handle failed payments."**

**Rahim (3): "The payment gateway gives us a ready-made SDK."**

**Round 2: everyone shows 5 → agreed estimate: 5 points**

# Velocity: How Much Fits in a Sprint?

## Velocity

Story points the team finishes in one sprint.

Example: Sprint 1 finished 16 points → plan about 16 for Sprint 2.

## Your First Sprint

- No history yet, so make a careful guess
- Commit to a little less than you think
- Measure, then adjust next sprint

**Only count stories that are fully Done. 90% done = 0 points.**

# Sprint Planning: Inputs, Steps, Output

### Inputs

- Ordered backlog
- Story points
- Team availability

### Steps

1. Agree the sprint goal
2. Pick top stories that fit capacity
3. Split stories into tasks
4. Members volunteer for tasks
5. Move stories to To Do

### Output

- Sprint goal
- Sprint backlog
- Task owners

**The Product Owner decides what matters most; the team decides how much it can take.**

# Worked Example: Sprint 1 Plan

> **Sprint goal: a customer can browse the menu, add dishes to a cart and pay online.**

| ID | Story | Priority | Points |
|---|---|---|---|
| US1 | Browse the menu | Must | 3 |
| US2 | Add items to a cart | Must | 5 |
| US3 | Pay online | Must | 5 |
| US5 | Manager updates menu items | Should | 3 |
| | **Total for a 2-week sprint** | | **16 points** |

US4 “Track my order” (Should, 8 points) does not fit, so it waits for Sprint 2.

# Breaking a Story into Tasks

> **US2: As a customer, I want to add items to a cart so that I can order several dishes. (5 points)**

| Task | Owner |
|---|---|
| Design the cart screen | Sadia |
| Build the add-to-cart function | Rahim |
| Save the cart in the database | Tanvir |
| Write unit tests for the cart | Nusrat |
| Review and merge the pull request | Whole team |

**Tasks take a day or less**

**Members volunteer for tasks**

**One story = one branch**

# Prepare Before Lab 4

- Install Git or GitHub Desktop on your laptop
- Check every member can access the team repo
- Estimate your Must stories with story points
- Write your sprint goal and Sprint 1 plan in /docs/sprint1.md

# Coming Up: Lab 4

## Running the Sprint and Scheduling

- Daily stand-up meetings
- Burndown charts
- Sprint review and retrospective
- Gantt chart and critical path

> **Bring your Sprint 1 plan: it is the input for Lab 4.**

# Key Takeaways

- Git records every change; GitHub shares it with the team
- One branch per story; merge through a reviewed pull request
- Story points measure size, not hours
- Planning poker turns disagreement into shared understanding
- Plan each sprint from the team’s real velocity

# Questions?